



Marwadi
University
Marwadi Chandarana Group

NAAC



MARWADI UNIVERSITY

FACULTY OF COMPUTER APPLICATIONS

Programming with Python & Problem Solving (05MC0107)

LAB MANUAL

Programming with Python & Problem Solving (05MC0107) Lab Manual (2026-27)

Program: MCA

Semester: 1

Name of the Student: SAMIR SAJIDBHAI SUMRA

Enrolment No.: 92600584103

INDEX

Lab	Program	Date	Remarks	Signature
1	Write a program to demonstrate basic input and output operations in Python.	17/08/2026		
2	Write a program to illustrate the use of different data types and type casting.	17/08/2026		
3	Write a program to perform arithmetic relational and logical operations using Python operators.	17/08/2026		
4	Write a program to demonstrate string operations including slicing formatting and built-in string functions.	17/08/2026		
5	Write a program to create and manipulate lists using indexing slicing and list comprehensions.	17/08/2026		
6	Write a program to illustrate the use of tuples and sets with basic operations.	17/08/2026		
7	Write a program to create a dictionary and demonstrate dictionary methods and iteration.	17/08/2026		
8	Write a program to explain mutable and immutable objects in Python.	17/08/2026		
9	Write a program to define and use user-defined functions with different types of arguments.	17/08/2026		
10	Write a program to demonstrate recursion using factorial or Fibonacci series.	17/08/2026		
11				
12				
13				
14				
15				
16				
17				
18				
19				
20				
21				
22				
23				
24				
25				
26				
27				
28				
29				



Marwadi
University
Marwadi Chandarana Group

NAAC



MARWADI UNIVERSITY

FACULTY OF COMPUTER APPLICATIONS

Programming with Python & Problem Solving (05MC0107)

LAB MANUAL

30				
31				
32				
33				
34				
35				
36				

Practical No. 1

Definition : Write a program to demonstrate basic input and output operations in Python.

Aim/ Objective:

- To understand how to accept input from the user in Python.
- To display output using the print() function.
- To perform basic operations on user-provided data.
- To understand basic Python syntax for input and output.

Source Code:

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print("\n--- User Details ---")
print("Name:", name)
print("Age:", age)
num1 = float(input("\nEnter first number: "))
num2 = float(input("Enter second number: "))
sum = num1 + num2
print("Sum of two numbers:", sum)
```

Output:

Enter your name: Samir

Enter your age: 20

--- User Details ---

Name: Samir

Age: 20

Enter first number: 25

Enter second number: 15

Sum of two numbers: 40.0

Conclusion:

The program successfully demonstrates the basic input and output operations in Python. It shows how to accept data from the user using `input()`, convert input into appropriate data types, store values in variables, perform a basic arithmetic operation, and display the results using `print()`

Practical No. 2

Definition: Write a program to illustrate the use of different data types and type casting.

Aim/ Objective:

- To understand different Python data types such as `int`, `float`, `str`, and `bool`.
- To learn how to check the type of a variable using `type()`.
- To understand type casting using `int()`, `float()`, `str()`, and `bool()`.
- To demonstrate conversion of data from one type to another.

Source Code:

```
a=10
print("integer",a,"type",type(a))
b=20.5
print("float",b,"type",type(b))
x=float(a)
print("float",x,"type",type(x))
y=str(a)
print("int to string",y,"type",type(y))
```

Output:

```
integer 10 type <class 'int'>
float 20.5 type <class 'float'>
float 10.0 type <class 'float'>
int to string 10 type <class 'str'>
```

Conclusion:

Thus, we successfully learned about different data types in Python and demonstrated type casting. We understood how to convert values from one data type to another using functions such as `int()`, `float()`, `str()`, and `bool()`.

Practical No. 3

Definition: Write a program to perform arithmetic relational and logical operations using Python operators.

Aim/Objectives:

- To understand and use arithmetic operators such as `+`, `-`, `*`, `/`, `%`, `//`, and `**`.
- To perform relational operations using `>`, `<`, `>=`, `<=`, `==`, and `!=`.
- To perform logical operations using `and`, `or`, and `not`.
- To understand how Python operators are used to perform calculations and comparisons.

Source Code:

```
num1=input("enter your number:")
print("your number is:",num1)
num2=input("enter your number:")
print("your number is:",num2)
addition=int(num1)+int(num2)
print("the addition is:",addition)
print("the subtraction is:",int(num1)-int(num2))
print("the multiplication is:",int(num1)*int(num2))
print("the division is:",int(num1)/int(num2))
print("the modulus is:",int(num1)%int(num2))
print("the floor division is:",int(num1)//int(num2))
print("the exponent is:",int(num1)**int(num2))
print("the average is:",(int(num1)+int(num2))/2)
a = True
b = False
print("Value of a:", a)
```

```
print("Value of b:", b)
print("\na AND b =", a and b)
print("a OR b =", a or b)
print("NOT a =", not a)
print("NOT b =", not b)
```

Output:

```
enter your number:22
your number is: 22
enter your number:33
your number is: 33
the addition is: 55
the subtraction is: -11
the multiplication is: 726
the division is: 0.6666666666666666
the modulus is: 22
the floor division is: 0
the exponent is: 199502557355935975909450298726667414302359552
the average is: 27.5
Value of a: True
Value of b: False

a AND b = False
a OR b = True
NOT a = False
NOT b = True
```

Conclusion: Thus, we successfully performed **arithmetic, relational, and logical operations** using Python operators. We learned how arithmetic operators perform calculations, relational operators compare values, and logical operators combine or reverse conditions.

Practical No. 4

Definition: Write a program to demonstrate string operations including slicing formatting and built-in string functions.

Aim/Objective:

- To understand and perform basic string operations in Python.
- To demonstrate string slicing to access specific characters.
- To use string formatting techniques.
- To apply built-in string functions for manipulating strings.
- To understand efficient handling and processing of strings in Python.

Source Code:

```
# Program to demonstrate string operations
```

```
# Input string
```

```
name = input("Enter your name: ")
```

```
# Slicing
```

```
print("\n--- String Slicing ---")
```

```
print("Original String :", name)
```

```
print("First 3 characters :", name[:3])
```

```
print("Last 3 characters :", name[-3:])
```

```
print("Characters from index 2 to 5 :", name[2:6])
```

```
# String Formatting
```

```
age = int(input("\nEnter your age: "))
```

```
print("\n--- String Formatting ---")
```

```
print("My name is {} and I am {} years old.".format(name, age))
```

```
print(f"My name is {name} and I am {age} years old.")
```

```
# Built-in String Functions
```

```
print("\n--- Built-in String Functions ---")
```

```
print("Upper Case :", name.upper())
print("Lower Case :", name.lower())
print("Length :", len(name))
print("Replace 'a' with '@' :", name.replace('a', '@'))
print("Starts with 'S' :", name.startswith('S'))
print("Ends with 'r' :", name.endswith('r'))
print("Is Alphabetic :", name.isalpha())
```

Output:

Enter your name: samir

--- String Slicing ---

Original String : samir

First 3 characters : sam

Last 3 characters : mir

Characters from index 2 to 5 : mir

Enter your age: 20

--- String Formatting ---

My name is samir and I am 20 years old.

My name is samir and I am 20 years old.

--- Built-in String Functions ---

Upper Case : SAMIR

Lower Case : samir

Length : 5

Replace 'a' with '@' : s@mir

Starts with 'S' : False

Ends with 'r' : True

Is Alphabetic : True

Conclusion:

Thus, we learned how to perform string slicing, formatting, and built-in string functions in Python.

We also understood how these operations help in efficient string manipulation.

Practical No. 5

Definition: Write a program to create and manipulate lists using indexing slicing and list comprehensions.

Aim/Objectives:

- To create and manipulate lists in Python.
- To access list elements using indexing.
- To extract elements using slicing.
- To create new lists using list comprehensions.
- To understand basic list manipulation techniques efficiently.

Source Code:

```
numbers=[10,20,30,40,50]
print("Original Numbers:",numbers)
print("Number First:",numbers[1])
print("Number Third:",numbers[2])
print("Last Number:",numbers[-1])
print("Last three elements:",numbers[2:5])
print("First three elements:",numbers[0:3])
print("First element:",numbers[0])
numbers.append(60)
print("After appened:",numbers)
numbers.remove(10)
print("After remove:",numbers)
numbers[0]=25
```

```
print("After changing the value:",numbers)
squares=[x*x for x in numbers]
print("Squares Numbers are:",squares)
even_numbers=[x for x in numbers if x %2==0]
print("Even Numbers are:",even_numbers)
```

Output:

Original Numbers: [10, 20, 30, 40, 50]
Number First: 20
Number Third: 30
Last Number: 50
Last three elements: [30, 40, 50]
First three elements: [10, 20, 30]
First element: 10
After appened: [10, 20, 30, 40, 50, 60]
After remove: [20, 30, 40, 50, 60]
After changing the value: [25, 30, 40, 50, 60]
Squares Numbers are: [625, 900, 1600, 2500, 3600]
Even Numbers are: [30, 40, 50, 60]

Conclusion:

To understand how to access, modify, and extract elements from a list using indexing and slicing, and to create new lists efficiently using list comprehensions.

Practical No. 6

Definition:

Write a program to illustrate the use of tuples and sets with basic operations.

Aim/Objectives:

- To understand the use of tuples and sets in Python.
- To create and initialize tuples and sets.

- To perform basic operations on tuples.
- To perform basic set operations such as union, intersection, and difference.
- To understand the importance of tuples and sets in Python.

Source Code:

```
# Tuple
```

```
numbers = (10, 20, 30, 40, 50)
print("Tuple:", numbers)
print("First element:", numbers[0])
print("Last element:", numbers[-1])
print("Sliced tuple:", numbers[1:4])
print("Length of tuple:", len(numbers))
print("Count of 20:", numbers.count(20))
print("Index of 30:", numbers.index(30))
set1 = {10, 20, 30, 40}
set2 = {30, 40, 50, 60}
print("\nSet 1:", set1)
print("Set 2:", set2)
set1.add(70)
print("After adding 70:", set1)
set1.remove(20)
print("After removing 20:", set1)
print("Union:", set1.union(set2))
print("Intersection:", set1.intersection(set2))
print("Difference:", set1.difference(set2))
```

Output:

Tuple: (10, 20, 30, 40, 50)

First element: 10

Last element: 50

Sliced tuple: (20, 30, 40)

Length of tuple: 5

Count of 20: 1

Index of 30: 2

Set 1: {40, 10, 20, 30}

Set 2: {40, 50, 60, 30}

After adding 70: {70, 40, 10, 20, 30}

After removing 20: {70, 40, 10, 30}

Union: {70, 40, 10, 50, 60, 30}

Intersection: {40, 30}

Difference: {10, 70}

Conclusion:

Thus, we learned how to create and manipulate tuples and sets using basic operations.

We also understood their differences and practical uses in Python.

Practical No. 7

Definition:

Write a program to create a dictionary and demonstrate dictionary methods and iteration.

Aim/Objectives:

- To understand the concept of dictionaries in Python.
- To create and initialize a dictionary.
- To perform various dictionary methods.
- To iterate through dictionary keys and values.
- To understand the use of dictionaries for storing key-value pairs.

Source Code:

```
# Create a dictionary
```

```
student = {
```

```
    "RollNo": 101,
```

```
"Name": "Samir",
"Course": "MCA",
"Marks": 85
}
print("Original Dictionary:", student)
# Accessing values
print("Student Name:", student["Name"])
# Dictionary methods
print("Keys:", student.keys())
print("Values:", student.values())
print("Items:", student.items())
# Add a new item
student["Grade"] = "A"
print("After adding Grade:", student)
# Update an item
student["Marks"] = 90
print("After updating Marks:", student)
# Remove an item
student.pop("Grade")
print("After removing Grade:", student)
# Iteration through dictionary
print("\nDictionary Elements:")
for key, value in student.items():
    print(key, ":", value)
```

Output:

```
Original Dictionary: {'RollNo': 101, 'Name': 'Samir', 'Course': 'MCA', 'Marks': 85}
Student Name: Samir
Keys: dict_keys(['RollNo', 'Name', 'Course', 'Marks'])
```

Values: dict_values([101, 'Samir', 'MCA', 85])

Items: dict_items([('RollNo', 101), ('Name', 'Samir'), ('Course', 'MCA'), ('Marks', 85)])

After adding Grade: {'RollNo': 101, 'Name': 'Samir', 'Course': 'MCA', 'Marks': 85, 'Grade': 'A'}

After updating Marks: {'RollNo': 101, 'Name': 'Samir', 'Course': 'MCA', 'Marks': 90, 'Grade': 'A'}

After removing Grade: {'RollNo': 101, 'Name': 'Samir', 'Course': 'MCA', 'Marks': 90}

Dictionary Elements:

RollNo : 101

Name : Samir

Course : MCA

Marks : 90

Conclusion:

Thus, we learned how to create and manipulate dictionaries using different methods and iteration. We also understood how to access and process key-value pairs efficiently in Python.

Practical No. 8

Definition:

Write a program to explain mutable and immutable objects in Python.

Aim/Objective:

- To understand the concept of mutable and immutable objects in Python.
- To identify mutable and immutable data types.
- To demonstrate modification of mutable objects.
- To understand why immutable objects cannot be changed directly.
- To differentiate between mutable and immutable objects with examples.

Source Code:

Original List: [10, 20, 30]

Modified List: [100, 20, 30]

Original Tuple: (10, 20, 30)

Original String: Samir

Modified String: Ravi

Output:

Original List: [10, 20, 30]

Modified List: [100, 20, 30]

Original Tuple: (10, 20, 30)

Original String: Samir

Modified String: Ravi

Conclusion:

Thus, we learned the difference between mutable and immutable objects in Python.

We also understood how changes affect mutable and immutable data types.

Practical No. 9

Definition:

. Write a program to define and use user-defined functions with different types of arguments.

Aim/Objective:

- To understand the concept of user-defined functions in Python.
- To define and call functions according to requirements.
- To demonstrate different types of function arguments.
- To use positional, keyword, default, and variable-length arguments.
- To understand the importance of functions in making programs modular and reusable.

Source Code:

```
def greet():  
    print("Hello, Welcome to Python!")  
def add(a, b):  
    print("Addition:", a + b)  
def student(name, course="MCA"):  
    print("Name:", name)  
    print("Course:", course)  
  
def details(name, age):  
    print("Name:", name)  
    print("Age:", age)  
def total(*numbers):  
    print("Total:", sum(numbers))  
greet()  
add(10, 20)  
student("Samir")  
student("Rahul", "BCA")  
details(age=22, name="Samir")  
total(10, 20, 30, 40)
```

Output:

```
Hello, Welcome to Python!  
Addition: 30  
Name: Samir  
Course: MCA  
Name: Rahul  
Course: BCA  
Name: Samir  
Age: 22
```

Total: 100

Conclusion:

Thus, we learned how to define and use user-defined functions with different types of arguments.

We also understood how functions make Python programs simple, organized, and reusable.

Practical No. 10

Definition:

Write a program to demonstrate recursion using factorial or Fibonacci series.

Aim/Objectives:

- To understand the concept of recursion in Python.
- To define and use a recursive function.
- To demonstrate recursion using factorial or Fibonacci series.
- To understand the base condition and recursive call.
- To learn how recursive functions solve problems step by step.

Source Code:

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)  
num = int(input("Enter a number: "))  
result = factorial(num)  
print("Factorial of", num, "is:", result)
```

Output:

Enter a number: 55

Factorial of 55 is:

12696403353658275925965100847566516959580321051449436762275840000000000000

Conclusion:

Thus, we learned how to implement recursion using factorial or Fibonacci series. We also understood the importance of the base condition and recursive calls.